REACH

Reinforcement-learning for Enhanced Arm Control in Humans

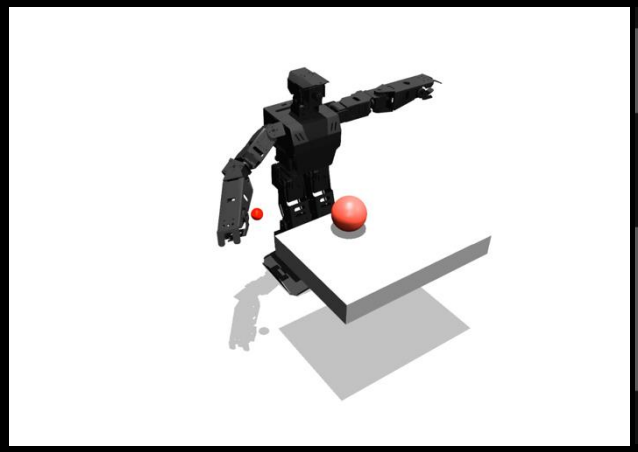
Members: TJ Davis, Lucas Larson, Clayton Ramsey, Victor Rodriguez

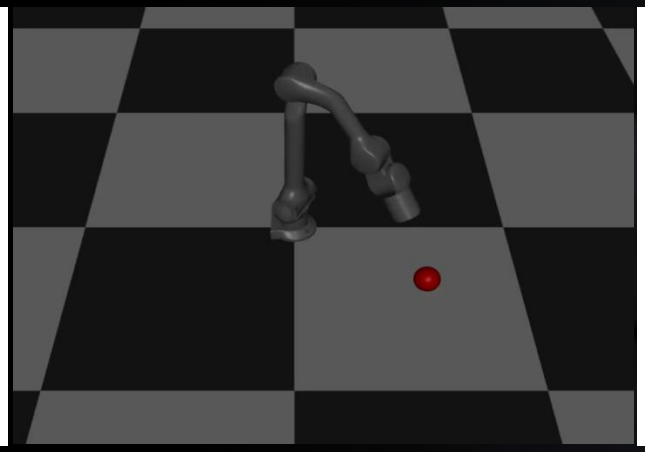
Clients: Dr. Zach Lerner & Dr. Carlo R. da Cunha

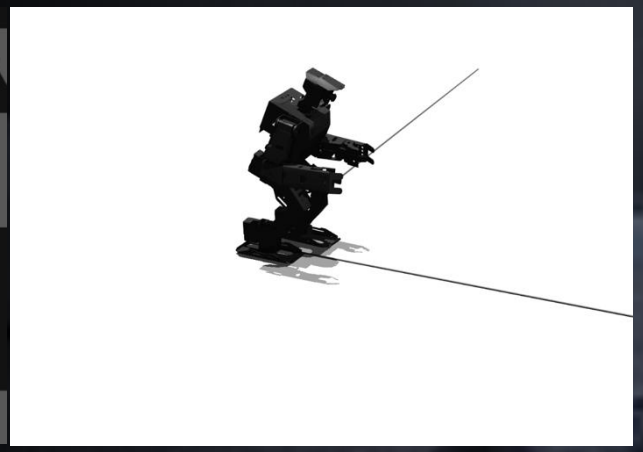
Mentor: Bailey Hall

Project Context

- Domain: Robotics simulation with reinforcement learning
- Value: faster iteration on policies before physical deployment







Sponsor & Client Need

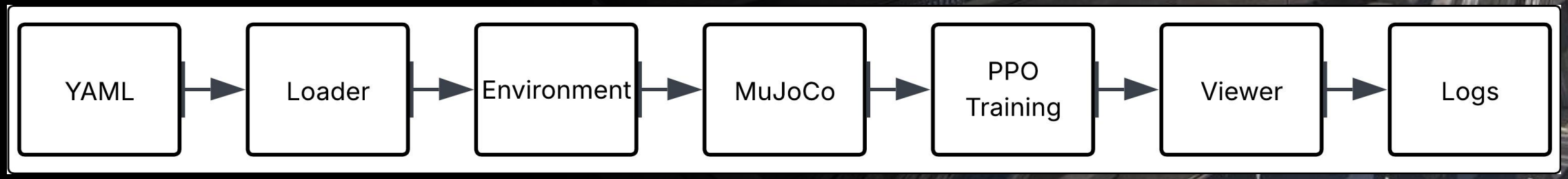
- Who: Biomechatronics Lab — Dr. Zach Lerner, Associate Professor, Mechanical Engineering, NAU & Dr Carlo Da Cunha, Assistant Professor, Electrical Engineering, NAU
- What they do: Research in prosthetics, exoskeletons, and robotic control systems
- Specific needs: Reproducible training and evaluation; unified workflow to switch scenes, models, and policies; evidence for analysis (logs, trajectories); cross-platform support
- Current gap: Ad-hoc pipelines, no unified way to switch tasks

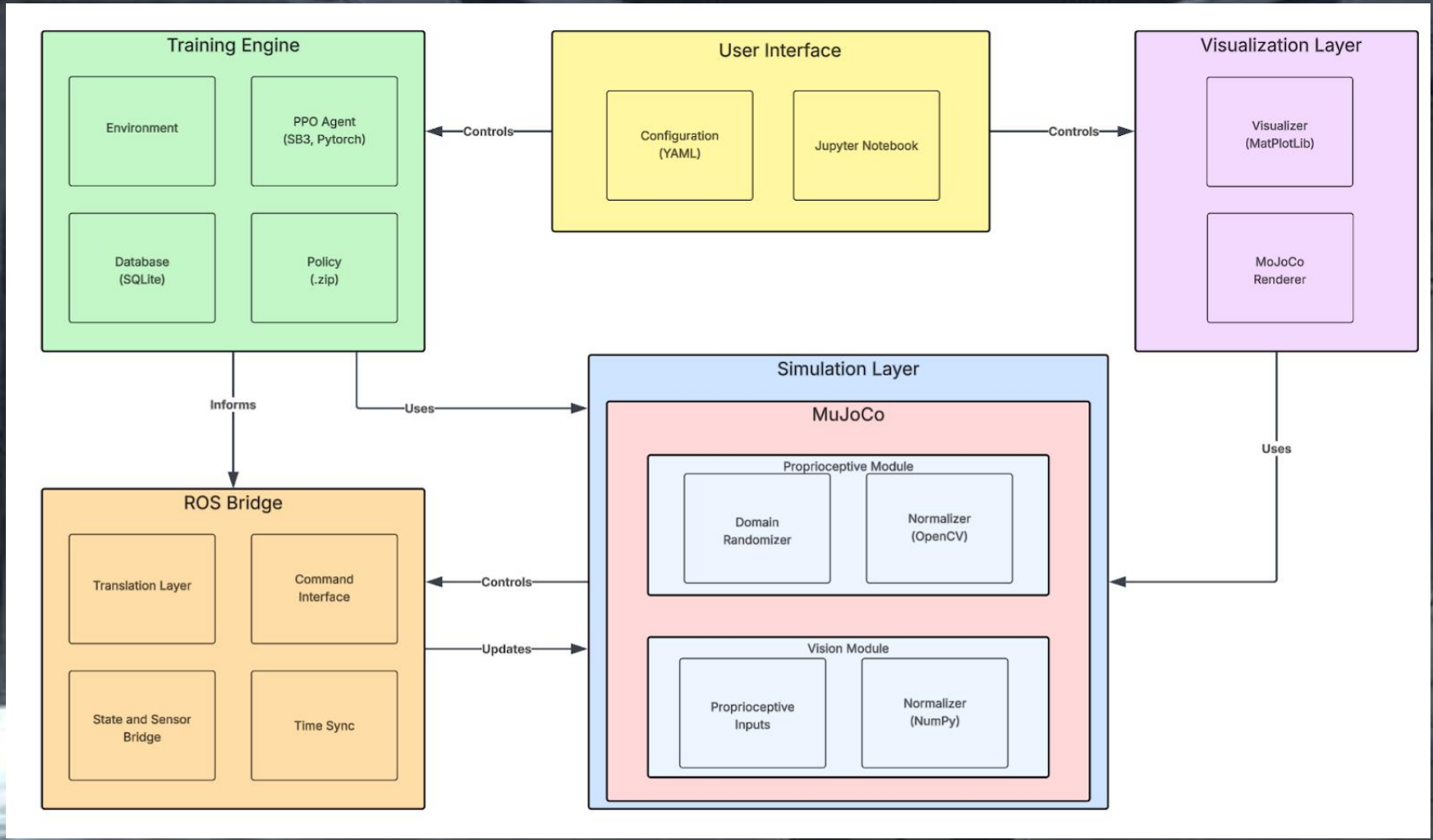
Problem Statement & Solution Vision

- Problem: Ad-hoc pipelines, hard to reproduce; no unified way to switch scenes, models, and policies
- Solution: REACH — YAML-driven platform for training, evaluation, and rendering
- Connection: Reproducibility → single config | Flexibility → change YAML paths | Speed → iterate in simulation first
- Value: One config file controls the entire workflow—switch tasks in minutes, not days.

Software Architecture Overview

- **Tech stack: Python • MuJoCo • Gymnasium • Stable-Baselines3 (PPO) • PyTorch • YAML**
- Modular structure: scenes/<scene_name>/ for models, policies, training scripts



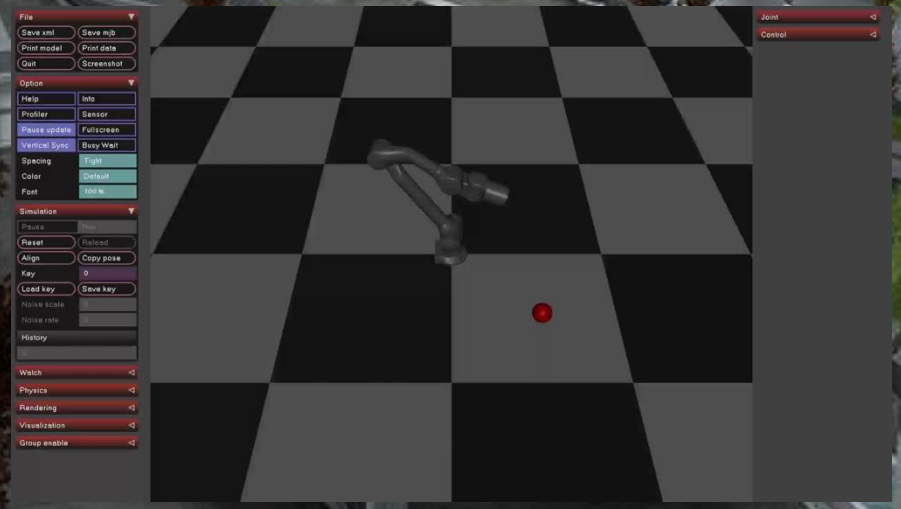

Training Engine
Environment
PPO Agent
(SB3, Pytorch)
Database
(SQLite)
Policy
(.zip)
Controls
User Interface
Configuration
(YAML)
Jupyter Notebook
Controls
Visualization Layer
Visualizer
(MatPlotLib)
MoJoCo
Renderer
Informs
Uses
Simulation Layer
MuJoCo
Proprioceptive Module
Domain
Randomizer
Normalizer
(OpenCV)
Vision Module
Proprioceptive
Inputs
Normalizer
(NumPy)
Uses
ROS Bridge
Translation Layer
Command
Interface
State and Sensor
Bridge
Time Sync
Controls
Updates

Key Components

- **YAML config — Defines scene, model XML, policy path, run settings**
- **Config loader — Validates and resolves paths**
- **MuJoCo model — Physics, actuators, geometry**
- **Gymnasium env — RL API: step, reset, observations, rewards**
- **PPO policy — Trained control policy from .zip**
- **Render pipeline — Rollouts, viewer, trajectories**

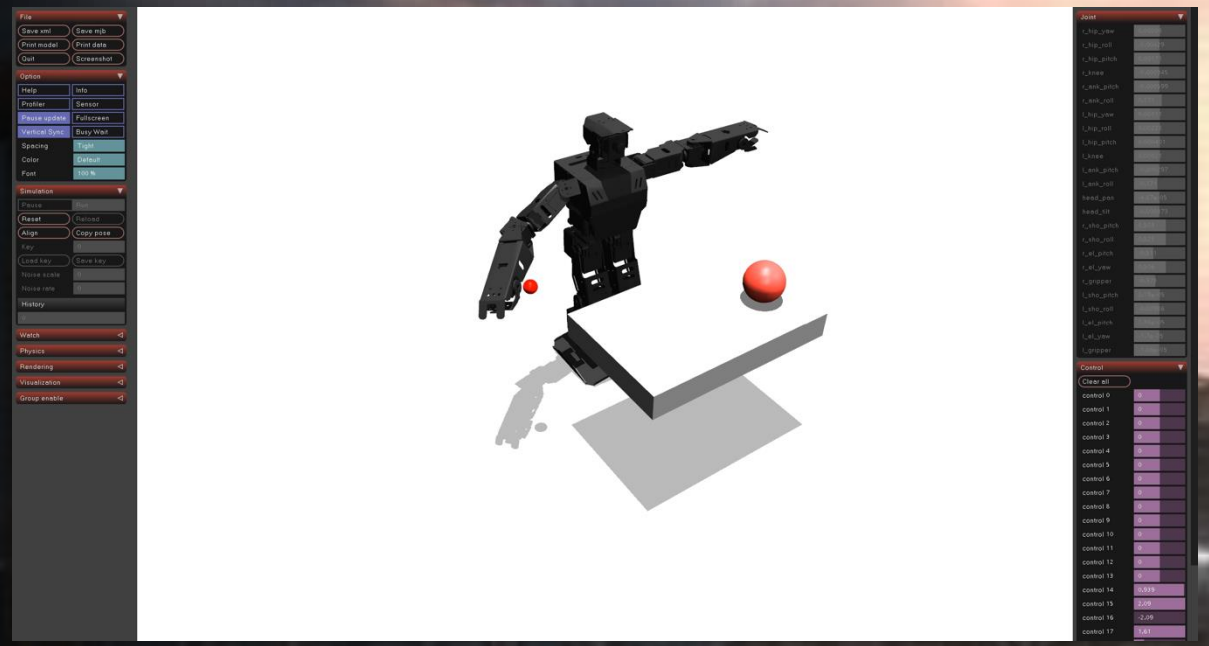
Design Decisions & Rationale

- MuJoCo — Industry standard for physics and visualization
- Gymnasium — RL compatibility; standard RL interface
- Stable-Baselines3 PPO — Stable, widely used for continuous control
- YAML configs — Reproducibility; no code changes to switch tasks
- Logs and trajectories — Evidence for analysis and evaluation



Alpha I Prototype: AINex Reach Task

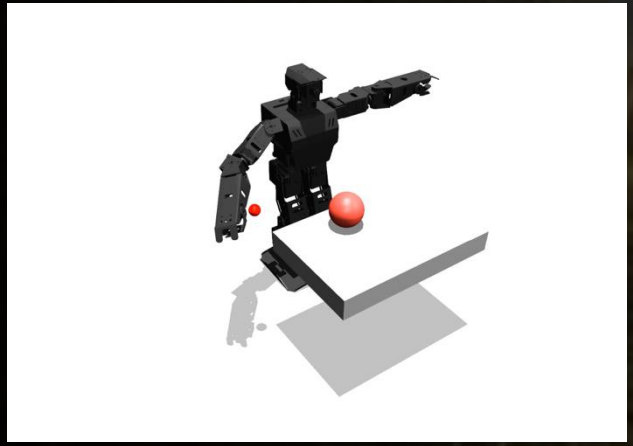
- Scene: scenes/ainex_reaching/
- Task: AINex arm reaches a ball on a table
- Reward: Distance-based and action smoothness
- Policy: PPO trained with parallel environments
- Alignment: This demo shows our full architecture in action, which is config-driven, modular, and reproducible



Alpha I Demonstration

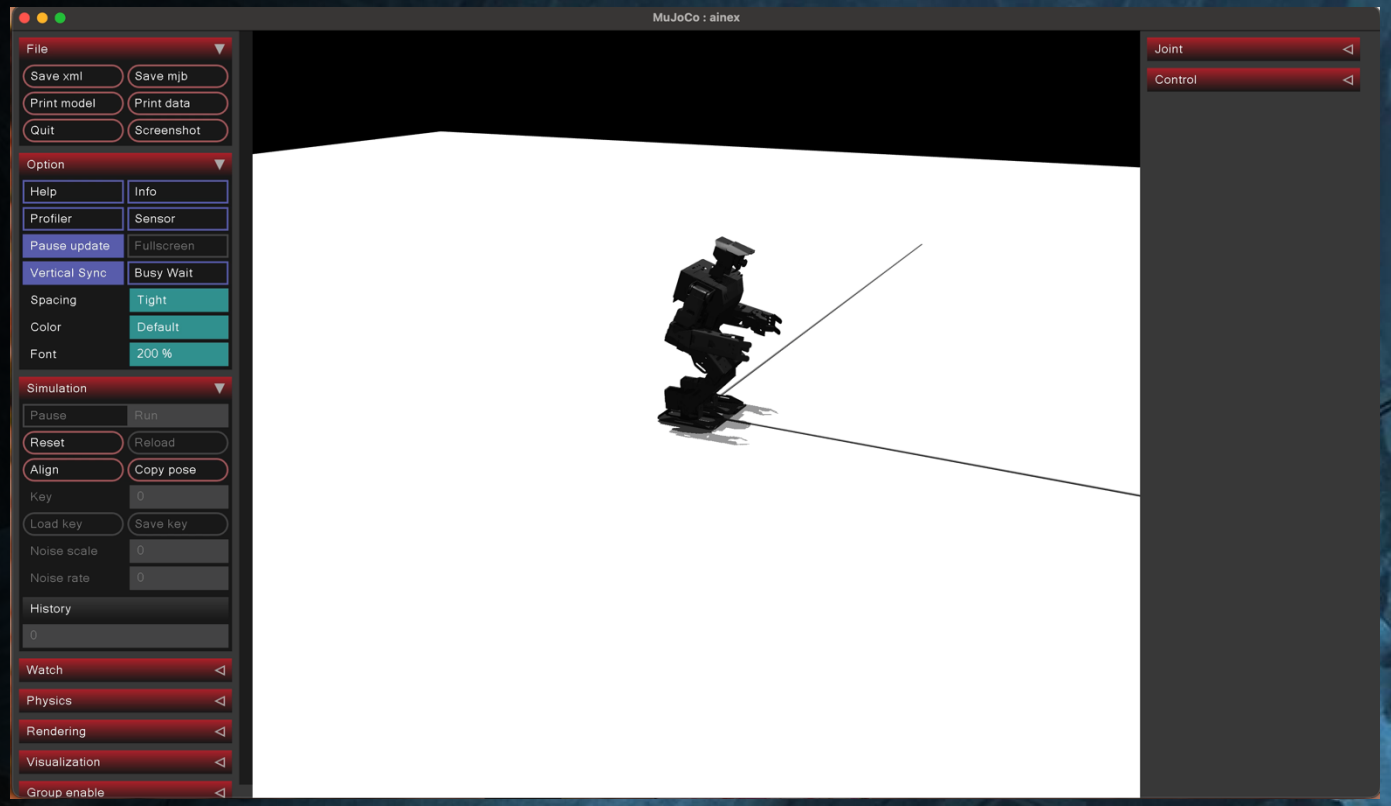
- Step 1: Config - config/ainex_reach.yaml selects scene, model, and policy
- Step 2: Command - mjpython scenes/ainex_reaching/training/ainex_reach_eval.py --config config/ainex_reach.yaml
- Step 3: Run – The policy loads, and the arm reaches the ball in the MuJoCo viewer
- Step 4: Output - Trajectories are saved in logs/trajectories/, plots, and logs

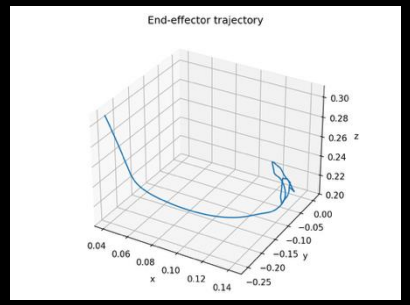
```yaml
scene:
  env_class: scenes.ainex_soccer.env:AINexReachEnv
  model_xml: scenes/ainex_soccer/models/ainex_stable.xml

policy:
  path: scenes/ainex_soccer/policies/ppo_ainex_reach.zip

run:
  episodes: 5
  max_seconds_per_ep: 20.0
  deterministic: true
  trace: true
```

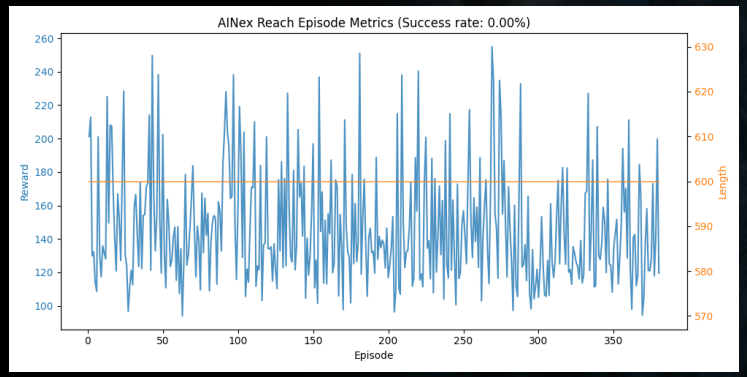


Working Functionality

- Evidence: Rendered rollout from trained policy, arm reaches ball, trajectory saved
- Outputs: Viewer, trajectory plots, logs
- Architecture alignment: Same flow as our design Config, Loader, Env, Policy, Viewer, then Logs



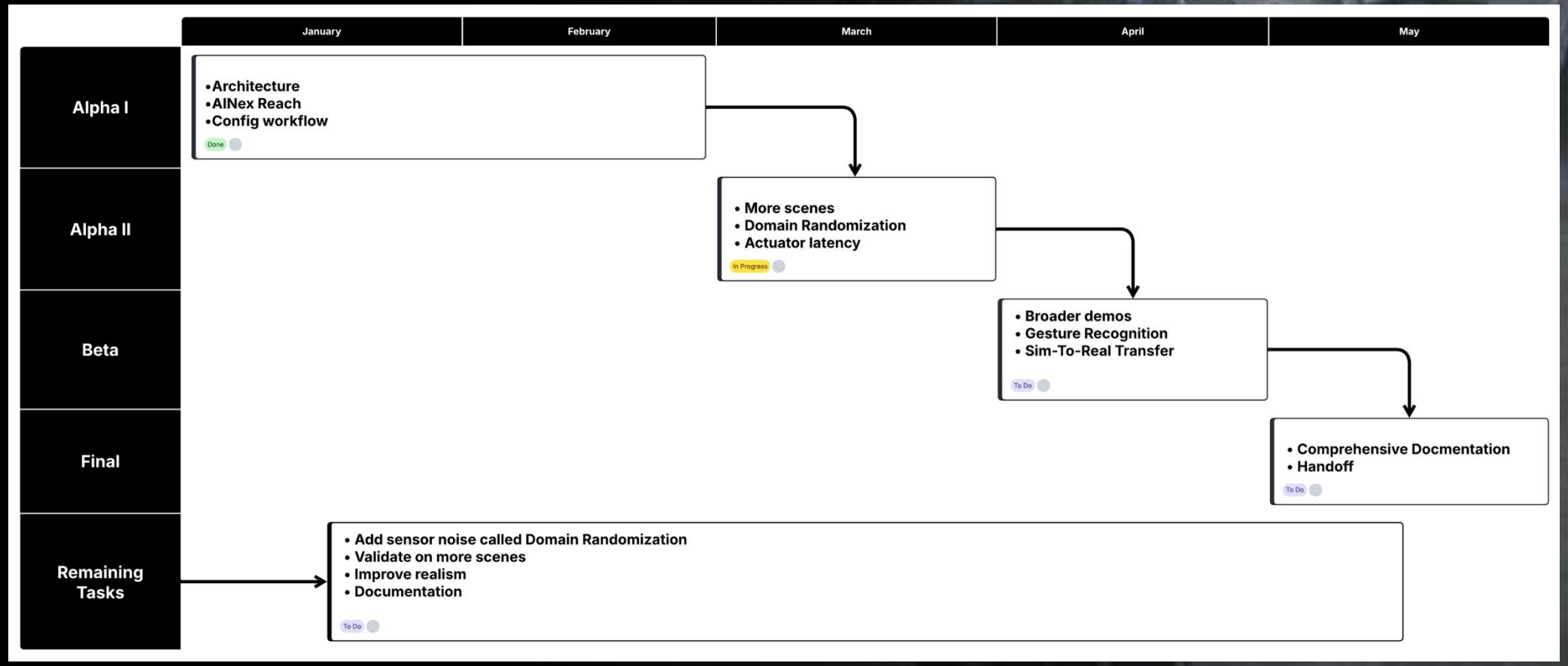




Challenges & Resolutions

- Obstacles: Sim-to-real gap, simulated realism, cross-platform differences
- How addressed: Documented limitations, planned noise, calibration, safety constraints, YAML for cross-platform config
- Open questions: Collaboration timeline with lab, priority of scenes vs realism, Beta scope
- Status: Architecture stable, Alpha I prototype validated

Plan & Schedule



Conclusion

- REACH delivers a unified RL simulation platform
- Architecture supports repeatable, configurable demos
- Alpha I proves the workflow end-to-end
- Ready to scale to more tasks and higher realism
- Next: Alpha II - more scenes, realism improvements, Beta preparation

Thank You